Object-Oriented Programming I

Lecture 2: Objects and Classes

Dr. Ghemougui Abdessettar

University of M'Sila – Faculty of Mathematics and Computer Science

October 6, 2026

Lecture plan

- What is an object?
- Object = State + Behavior
- What is a class?
- Class vs. object
- Defining classes in C++

Object

- An object is anything to which we can associate certain **properties** and **behavior**.
- From the analyst's point of view: an object represents an entity (real or abstract) with a well-defined role in the problem domain.
- From the programmer's point of view: an object is a data structure on which we can perform a well-defined set of operations.

Object

According to *Grady Booch*, an object has a state, a behavior, and an identity:

- The **state** describes an object's properties at a given moment.
- The **behavior** describes the actions the object can perform.
- The **identity** is the property that distinguishes one object from another (e.g., a unique variable name).

Phone
battery : 80% power : on used_storage : 43 GB wi-fi : connected
install_app run_app charge connect_wifi power_on_off

State and Behavior Are Related

Behavior operates on state and can change it.

- The **install_app** operation changes the **used_storage** state.
- The **run_app** operation changes the **battery** state.
- The **charge** operation changes the **battery** state.

Phone
battery : 80% power : on used_storage : 43 GB wi-fi : connected
install_app run_app charge connect_wifi power_on_off

Each Object Has Its Own State

Phone 1
battery : 80% power : on used_storage : 43 GB wi-fi : connected
install_app run_app charge connect_wifi power_on_off

Phone 2
battery : 35% power : on used_storage : 81 GB wi-fi : not connected
install_app run_app charge connect_wifi power_on_off

State Changes Should Have an Explanation

If the battery goes from 75% to 80% while *not* charging, that is an unexpected state change.

Principle

An object's state should change only as a consequence of its behavior. The object itself should control how its state changes — not any arbitrary part of the program.

State → Attributes

State is represented by **attributes** (fields or instance variables):

Phone
battery : Integer
power : Boolean
used_storage : Integer
wifi_on : Boolean
wifi_net : String

```
[C++]C++ int battery; bool power; int used_storage; bool wifi_on; char wifi_net[20];
```

Behavior → Methods

Behavior is represented by **member functions** or **methods**:

Phone
charge()
powerOnOff()
runApp()
installApp()

[C++]C++ void charge() ...
void powerOnOff() ...
...

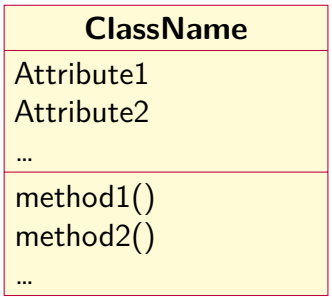
Class

- An abstraction of the attributes (characteristics), operations, relationships, and semantics common to a set of objects.
- Thus, a class represents the set of objects that share a common structure and behavior. All of them are **instances** of the same class.

Class

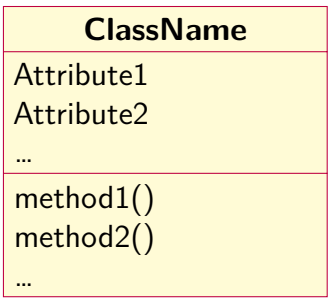
Parts of the definition

- **Class identifier**: name
- **Properties**
  - **Attributes** or *variables*: data needed to describe the objects (*instances*) created from the class.
    - The combination of their values determines the state of an object.
  - **Operations, methods, services**: actions that an object knows how to perform.
  - **Roles**: relationships that a class establishes with other classes.



Class

Parts of the definition



[C++]C++ class ClassName private: Type1 attribute1; Type2 attribute2; ... public: void method1(...)... void method2(...)... ... ;

Objects vs. Classes

Which of the following are objects and which are classes?

- Student
- Ali
- Tesla
- Car
- Smartphone
- iPhone 18 Pro

Objects vs. Classes

Classes

- Student
- Car
- Smartphone

Objects

- Ali $\rightarrow$ instance of Student
- Tesla $\rightarrow$ instance of Car
- iPhone 18 Pro $\rightarrow$ instance of Smartphone

Objects vs. Classes

Item	Type	Explanation
Student	Class	Template describing all students
Ali	Object	Instance of `Student`
Tesla	Object	Instance of `Car`
Car	Class	Template describing all cars
Smartphone	Class	Template describing all smartphones
iPhone 18 Pro	Object	Instance of `Smartphone`

Objects vs. Classes

Principle

- Something is a class if it can have instances.
- Something is an object if it is a specific individual that belongs to a class and shares that class's characteristics.

Objects vs. Classes: Programming View

From a programming perspective:

- Class $\approx$ Type
- Object $\approx$ Value of that type, stored in a variable or constant

`int x = 5;` $\rightarrow$ `x` is an instance of the type `int`

`Student ali;` $\rightarrow$ `ali` is an object of class `Student`

Declaring a Class in C++

```
class ClassName {
access_specifier:
    // attributes
    // methods
};   // <-- semicolon is mandatory
```

- `class` is the keyword that introduces a class.
- The body is enclosed in braces `{ }`.
- The access specifiers are **public**, **protected**, or **private**.
- The declaration **must end with a semicolon**.

Access Specifiers

Specifier	Accessible from
`private`	Inside the class only
`protected`	The class and its derived classes
`public`	Anywhere

Remark

In a `class`, members are `private` by default. In a `struct`, they are `public` by default.

A First Example

```
#include <stdio.h>
class Point {
public: // access specifier
  // attributes
  float x;
  float y;
  // methods
  void print() { printf("X= %.2f Y=%.2f\n", x, y); }
};
```

Creating and Using Objects

```
1 int main() {
2   Point p;
3   p.x = 100;
4   p.y = 22;
5   p.print();
6 }
```

- `Point` is the class and `p` is an object (or instance) of it.
- `p.x = 100;` changes the attribute `x` of the object `p`.
- `p.print();` calls the **method** `print` of the object `p`.

Changing the Access Specifier

```
#include <stdio.h>
class Point {
private:
  float x;
  float y;
public:
  void print() { printf("X= %.2f Y=%.2f\n", x, y); }
};

int main() {
  Point p;
  p.x = 100; // ERROR: x is private
  p.y = 22;  // ERROR: y is private
  p.print();
}
```

Changing the Access Specifier

```
#include <stdio.h>

int main() {
  Point p;
  p.x = 100; // ERROR: x is private
  p.y = 22;  // ERROR: y is private
  p.print();
}
```

Question

If the attributes x and y are **private**, how can we change them?

A Word on Encapsulation

Idea

`x` and `y` are part of the object's **state** and should only be manipulated by the object's **behavior**.

- Add the methods `setX()` and `setY()` to the class `Point`.
- Add a method `setPosition()` to the class `Point`.

A Word on Encapsulation

```
1  class Point {
2  private:
3    float x;
4    float y;
5  public:
6    void setPosition(float a, float b) {
7      x = a;
8      y = b;
9    }
10   void print() { printf("X= %.2f Y=%.2f\n", x, y); }
11 };
```

A Word on Encapsulation

```
1 int main() {
2   Point p;
3   p.setPosition(100, 22); // use a public method to set private attributes
4   p.print();
5 }
```

Idea

x and y are part of the object's **state** and should only be manipulated by the object's **behavior**. We updated the class so that the point's position (**state**) can only be updated through the public method **setPosition**.
This is a fundamental principle of OOP called Encapsulation. See the next lecture for more details on this concept.

Common Mistakes When Defining Classes

- Forgetting the semicolon after the closing brace.
- Accessing a `private` member from outside the class.
- Confusing the class (the blueprint) with the object (the instance).